python questions

150 Python Interview Questions for Freshers 🐍

1. Python Fundamentals (1–20)

1. What is Python?
2. Who developed Python?
3. What are the main features of Python?
4. What are the advantages of Python?
5. What are the limitations of Python?
6. Is Python a compiled or interpreted language?
7. What is dynamic typing in Python?
8. What is strong typing in Python?
9. What are variables in Python?
10. What are the rules for naming variables?
11. What are Python keywords?
12. What are identifiers in Python?
13. What are the different built-in data types in Python?
14. What is the difference between mutable and immutable objects?
15. What is type casting in Python?
16. What is the difference between implicit and explicit type conversion?
17. What is the difference between `==` and `is`?
18. What is the purpose of indentation in Python?

19. What is PEP 8?
20. What is the difference between a syntax error and a runtime error?

---

2. Operators and Control Flow (21–35)

1. What are the different types of operators in Python?
2. What is the difference between `/` and `//`?
3. What is the difference between `==` and `!=`?
4. What are logical operators in Python?
5. What are bitwise operators?
6. What is the difference between `and` and `&`?
7. What is the difference between `or` and `|`?
8. What is operator precedence?
9. What is the difference between `break`, `continue`, and `pass`?
10. What is the difference between `if`, `elif`, and `else`?
11. What is the difference between a `for` loop and a `while` loop?
12. What is the purpose of the `range()` function?
13. What is a nested loop?
14. What is the difference between `for` and `else` in Python?
15. What are truthy and falsy values in Python?

---

3. Strings (36–50)

1. What is a string in Python?
2. Are strings mutable or immutable?
3. What is string indexing?
4. What is string slicing?
5. What is negative indexing in strings?

6. What is the difference between `find()` and `index()`?
7. What is the difference between `split()` and `join()`?
8. What is the difference between `strip()`, `lstrip()`, and `rstrip()`?
9. What is the difference between `replace()` and `split()`?
10. What are f-strings in Python?
11. What is the difference between `upper()`, `lower()`, and `title()`?
12. How can you reverse a string in Python?
13. How do you check whether a string is a palindrome?
14. What is string interning?
15. What is the difference between `isalpha()`, `isdigit()`, and `isalnum()`?

---

4. Lists, Tuples, Sets, and Dictionaries (51–75)

1. What is a list in Python?
2. What is a tuple in Python?
3. What is a set in Python?
4. What is a dictionary in Python?
5. What is the difference between a list and a tuple?
6. What is the difference between a list and a set?
7. What is the difference between a set and a dictionary?
8. Why are tuples considered immutable?
9. Can a tuple contain a list?
10. Can a list be used as a dictionary key?
11. What is the difference between `append()` and `extend()`?
12. What is the difference between `remove()`, `pop()`, and `del`?
13. What is the difference between `sort()` and `sorted()`?
14. How do you remove duplicates from a list?

15. What is list comprehension?
16. What is dictionary comprehension?
17. What is a nested list?
18. What is a nested dictionary?
19. What is the difference between `dict[key]` and `dict.get(key)`?
20. What are `keys()`, `values()`, and `items()`?
21. What is unpacking in Python?
22. What are set union, intersection, and difference?
23. What is the difference between shallow copy and deep copy?
24. What is the difference between assignment and copying a list?
25. What is dictionary insertion order in Python?

---

5. Functions (76–90)

1. What is a function in Python?
2. What is the difference between parameters and arguments?
3. What is the difference between `print()` and `return`?
4. What happens when a function has no return statement?
5. What are default arguments?
6. What are keyword arguments?
7. What are positional arguments?
8. What are `args` and `*kwargs`?
9. What is a lambda function?
10. What is recursion?
11. What is a higher-order function?
12. What is a closure in Python?
13. What is a decorator?

14. What is variable scope in Python?
15. What is the difference between `global` and `nonlocal`?

6. Object-Oriented Programming (91–110)

1. What is object-oriented programming?
2. What is a class in Python?
3. What is an object in Python?
4. What is the difference between a class and an object?
5. What is the purpose of `self`?
6. What is the purpose of `__init__()`?
7. What are instance variables?
8. What are class variables?
9. What are instance methods?
10. What are class methods?
11. What are static methods?
12. What are the four pillars of OOP?
13. What is encapsulation?
14. What is abstraction?
15. What is inheritance?
16. What are the different types of inheritance?
17. What is multiple inheritance?
18. What is polymorphism?
19. What is the difference between method overloading and method overriding?
20. What is the purpose of `super()`?

7. Exception Handling, Files, and Modules (111–125)

1. What is exception handling in Python?
2. What is the difference between an error and an exception?
3. What is the purpose of `try` and `except`?
4. What is the purpose of `else` in exception handling?
5. What is the purpose of `finally`?
6. What is the purpose of the `raise` keyword?
7. How do you create a custom exception?
8. What is the difference between `SyntaxError`, `TypeError`, and `ValueError`?
9. What are the different file opening modes in Python?
10. What is the difference between `read()`, `readline()`, and `readlines()`?
11. What is the purpose of the `with` statement when working with files?
12. What is the difference between a module and a package?
13. What is the purpose of `import` in Python?
14. What is the purpose of `if __name__ == "__main__":`?
15. What is the difference between `pip` and a virtual environment?

---

8. Advanced Python (126–140)

1. What is an iterable in Python?
2. What is an iterator in Python?
3. What is the difference between an iterable and an iterator?
4. What is a generator in Python?
5. What is the difference between `yield` and `return`?
6. What is a generator expression?
7. What are magic or dunder methods?
8. What is the difference between `__str__()` and `__repr__()`?
9. What is operator overloading?

10. What is a context manager?
11. How does Python manage memory?
12. What is garbage collection?
13. What is the Global Interpreter Lock (GIL)?
14. What is the difference between multithreading and multiprocessing?
15. What is the difference between shallow binding and deep copying of nested objects?

---

9. Coding and Practical Questions (141–150)

1. Write a Python program to reverse a string.
2. Write a program to check whether a number is prime.
3. Write a program to check whether a string is a palindrome.
4. Write a program to find the factorial of a number.
5. Write a program to generate the Fibonacci series.
6. Write a program to find the second-largest element in a list.
7. Write a program to count the frequency of each character in a string.
8. Write a program to remove duplicates from a list without using a set.
9. Write a program to check whether two strings are anagrams.
10. Write a Python program to connect to MySQL and perform CRUD operations.

ANSWERS

1. Python Fundamentals (1–20)

1. What is Python?

Python is a high-level, interpreted, general-purpose programming language. It is widely known for its clean syntax, which closely resembles plain English, making it

very readable and easy to learn.

2. Who developed Python?

Python was developed by **Guido van Rossum**, and it was first released in 1991.

3. What are the main features of Python?

- `Easy to Learn and Read:` Clean syntax emphasizes code readability.
- `Interpreted:` Code is executed line by line, making debugging easier.
- `Dynamically Typed:` Variable types are determined at runtime rather than explicitly declared.
- `Object-Oriented & Procedural:` Supports multiple programming paradigms.
- `Extensive Libraries:` Offers a vast standard library and thousands of third-party packages via PyPI.

4. What are the advantages of Python?

- High productivity and fast development due to concise code and readability.
- Massive ecosystem for web development, data science, artificial intelligence, and automation.
- Platform-independent (runs on Windows, macOS, Linux).
- Large, active community with abundant documentation and tutorials.

5. What are the limitations of Python?

- **Execution Speed:** Being an interpreted language, it is generally slower than compiled languages like C or C++.
- **Memory Consumption:** It can use more memory compared to lower-level languages.
- **Global Interpreter Lock (GIL):** Limits true multi-threaded CPU-bound parallelism in standard CPython.
- **Mobile Development:** Not widely used for native mobile application development.

6. Is Python a compiled or interpreted language?

Python is technically **both**. First, the source code (`.py`) is compiled into intermediate bytecode (`.pyc`). Then, that bytecode is executed line by line by the Python Virtual Machine (PVM), which acts as an interpreter.

7. What is dynamic typing in Python?

Dynamic typing means you do not need to explicitly declare a variable's data type when creating it. Python determines the type automatically at runtime based on the value assigned, and a variable can change its data type later.

8. What is strong typing in Python?

Strong typing means Python does not perform implicit type conversion between incompatible types in a way that causes unexpected data loss. For example, you cannot directly add a string and an integer (`"5" + 2`) without explicit conversion.

9. What are variables in Python?

Variables are containers or symbolic names used to store data values in memory. In Python, a variable is created the moment you assign a value to it using the `=` operator.

10. What are the rules for naming variables?

- Must start with a letter (A-Z, a-z) or an underscore (`_`).
- Cannot start with a number.
- Can only contain alpha-numeric characters and underscores (`A-z`, `0-9`, and `_`).
- Are case-sensitive (`age`, `Age`, and `AGE` are distinct).
- Cannot be a reserved Python keyword.

11. What are Python keywords?

Keywords are reserved words with predefined meanings in Python (e.g., `if`, `else`, `while`, `def`, `class`, `return`). They are part of the language syntax and cannot be used as variable names or identifiers.

12. What are identifiers in Python?

Identifiers are user-defined names given to variables, functions, classes, modules, or other objects to uniquely identify them in the code.

13. What are the different built-in data types in Python?

- **Numeric:** `int`, `float`, `complex`
- **Sequence:** `str`, `list`, `tuple`, `range`
- **Mapping:** `dict`
- **Set:** `set`, `frozenset`
- **Boolean:** `bool`
- **Binary:** `bytes`, `bytearray`, `memoryview`

14. What is the difference between mutable and immutable objects?

- **Mutable objects** can be modified *in place* after creation (e.g., `list`, `dict`, `set`).
- **Immutable objects** cannot be changed once created; any modification creates a brand-new object in memory (e.g., `int`, `float`, `str`, `tuple`).

15. What is type casting in Python?

Type casting is the process of converting a variable from one data type to another using built-in constructor functions like `int()`, `float()`, `str()`, or `list()`.

16. What is the difference between implicit and explicit type conversion?

- **Implicit conversion** is handled automatically by Python without user intervention (e.g., adding an integer and a float results in a float).
- **Explicit conversion** (type casting) is performed manually by the programmer using built-in functions (e.g., converting the string `"10"` to an integer using `int("10")`).

17. What is the difference between `==` and `is`?

- `==` compares the **values** of two objects to check if they are equal.
- `is` compares the **memory addresses** (identity) of two objects to check if they point to the exact same object in memory.

18. What is the purpose of indentation in Python?

Unlike languages that use curly braces `{}` or keywords to define blocks of code, Python uses indentation (spaces or tabs) to define scope and control flow (such as inside loops, functions, and conditional statements). Proper indentation is syntactically mandatory.

19. What is PEP 8?

PEP 8 (Python Enhancement Proposal 8) is the official style guide and set of recommendations for writing clean, readable, and consistent Python code (e.g., using 4 spaces per indentation level and `snake_case` for function/variable names).

20. What is the difference between a syntax error and a runtime error?

- **Syntax error** occurs during parsing when code violates Python's grammatical rules, preventing the program from running at all.
- **Runtime error** occurs during execution *while* the program is running, even though the syntax is valid (e.g., dividing by zero or trying to access a file that doesn't exist).

---

2. Operators and Control Flow (21–35)

1. What are the different types of operators in Python?

Python supports several types of operators:

- **Arithmetic operators:** `+`, , , `/`, `//`, `%`, `*`
- **Comparison (Relational) operators:** `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Logical operators:** `and`, `or`, `not`
- **Bitwise operators:** `&`, `|`, `^`, `~`, `<<`, `>>`

- **Assignment operators:** `=`, `+=`, `=`, `=`, etc.
- **Identity operators:** `is`, `is not`
- **Membership operators:** `in`, `not in`

2. What is the difference between `/` and `//`?

- `/` is standard division, which always returns a float (e.g., `5 / 2 = 2.5`).
- `//` is floor division, which performs division and rounds down the result to the nearest integer (or whole number) less than or equal to the actual result (e.g., `5 // 2 = 2`, and `5 // 2 = -3`).

3. What is the difference between `==` and `!=`?

- `==` checks if two values are **equal**, returning `True` if they match.
- `!=` checks if two values are **not equal**, returning `True` if they are different.

4. What are logical operators in Python?

Logical operators are used to combine conditional statements:

- `and`: Returns `True` if both operands are true.
- `or`: Returns `True` if at least one operand is true.
- `not`: Reverses the boolean value (returns `True` if the operand is false, and vice versa).

5. What are bitwise operators?

Bitwise operators perform operations on integers at the binary (bit) level:

- `&` **(AND)**: Sets each bit to 1 if both bits are 1.
- `|` **(OR)**: Sets each bit to 1 if one of two bits is 1.
- `^` **(XOR)**: Sets each bit to 1 if only one of two bits is 1.
- `~` **(NOT)**: Inverts all the bits.
- `<<` **(Left shift)**: Shifts bits to the left by pushing zeros in from the right.
- `>>` **(Right shift)**: Shifts bits to the right.

6. What is the difference between `and` and `&`?

- `and` is a logical operator used for boolean evaluation. It supports **short-circuit evaluation** (if the first operand is false, it doesn't evaluate the second).
- `&` is a bitwise operator used for binary operations on integers. It can also be used as a logical/set operator for booleans or sets, but it **does not short-circuit** (both sides are always evaluated).

7. What is the difference between `or` and `|`?

- `or` is a logical operator with **short-circuit evaluation** (if the first operand is true, it returns it immediately without checking the second).
- `|` is a bitwise OR operator (and also used for set union or merging dictionaries in modern Python) that **evaluates both sides** without short-circuiting.

8. What is operator precedence?

Operator precedence defines the order in which operators are evaluated in an expression. For example, multiplication and division have higher precedence than addition and subtraction. Parentheses `()` can be used to explicitly override precedence and dictate the order of evaluation.

9. What is the difference between `break`, `continue`, and `pass`?

- `break`: Completely terminates and exits the loop it is inside.
- `continue`: Skips the rest of the current iteration and immediately jumps to the next iteration of the loop.
- `pass`: A null statement that acts as a placeholder; it does nothing when executed, used when syntax requires a statement but no code needs to run yet.

10. What is the difference between `if`, `elif`, and `else`?

- `if`: Evaluates a condition first; if true, executes its block.
- `elif` (short for "else if"): Evaluates alternative conditions sequentially only if all previous `if`/`elif` conditions were false.

- `else`: Acts as a catch-all block that executes only if all preceding `if` and `elif` conditions evaluate to false.

11. What is the difference between a `for` loop and a `while` loop?

- `for` **loop** is used for definite iteration—running a block of code a specific number of times or iterating over elements in a sequence (like a list, string, or range).
- `while` **loop** is used for indefinite iteration—running a block of code repeatedly as long as a specified boolean condition remains `True`.

12. What is the purpose of the `range()` function?

The `range()` function generates a sequence of numbers. It is most commonly used in `for` loops to iterate a specific number of times. It takes up to three arguments: `range(start, stop, step)`.

13. What is a nested loop?

A nested loop is a loop inside another loop (e.g., a `for` loop inside another `for` loop). The inner loop executes completely for every single iteration of the outer loop.

14. What is the difference between `for` and `else` in Python?

Python allows an `else` block to be attached to a `for` loop. The code inside the `else` block runs **only** if the loop completes all its iterations normally **without** encountering a `break` statement.

15. What are truthy and falsy values in Python?

In Python, any object can be evaluated in a boolean context:

- **Falsy values** evaluate to `False`. Examples include `None`, `False`, numeric zeros (`0`, `0.0`, `0j`), empty sequences/collections (`""`, `[]`, `()`, `{}`), and empty sets.
- **Truthy values** evaluate to `True`. This includes any object that is not defined as falsy (such as non-empty strings, non-zero numbers, and populated collections).

---

3. Strings (36–50)

1. What is a string in Python?

A string is a sequence of characters enclosed within single quotes (`'...'`), double quotes (`"..."`), or triple quotes (`'''...'''` or `"""..."""`). It is used to store and represent textual data in Python.

2. Are strings mutable or immutable?

Strings are **immutable**. Once a string is created, its contents cannot be changed or modified in place. Any operation that modifies a string actually creates and returns a brand-new string object in memory.

3. What is string indexing?

String indexing refers to accessing individual characters within a string based on their position (index). Python uses zero-based indexing, meaning the first character is at index `0`, the second at `1`, and so on.

4. What is string slicing?

String slicing is a technique used to extract a portion (substring) of a string using the syntax `[start:stop:step]`.

- `start`: The beginning index (inclusive).
- `stop`: The ending index (exclusive).
- `step`: The interval or stride between characters (defaults to 1).

5. What is negative indexing in strings?

Negative indexing allows you to access characters from the end of the string rather than the beginning. The last character has an index of `-1`, the second-to-last is `-2`, and so on, making it easy to reference elements from the back without knowing the string's length.

6. What is the difference between `find()` and `index()`?

Both methods search for a substring within a string:

- `find()` returns the lowest index of the substring if found; if not found, it gracefully returns `-1`.
- `index()` works similarly but raises a `ValueError` if the substring is not found.

7. What is the difference between `split()` and `join()`?

- `split()` takes a string and breaks it apart into a **list of substrings** based on a specified separator (default is whitespace).
- `join()` takes an iterable sequence of strings (like a list) and **concatenates them into a single string**, using the calling string as a separator.

8. What is the difference between `strip()`, `lstrip()`, and `rstrip()`?

- `strip()` removes leading (left) and trailing (right) whitespace or specified characters.
- `lstrip()` removes characters from the **left (leading)** side only.
- `rstrip()` removes characters from the **right (trailing)** side only.

9. What is the difference between `replace()` and `split()`?

- `replace()` substitutes occurrences of a specified old substring with a new substring, returning a modified string.
- `split()` breaks a string into multiple pieces, returning a **list** of substrings based on a delimiter.

10. What are f-strings in Python?

Introduced in Python 3.6, f-strings (formatted string literals) are a concise and readable way to embed variables and expressions inside strings. They are prefixed with `f` or `F` and use curly braces `{}` to evaluate variables or expressions directly at runtime.

11. What is the difference between `upper()`, `lower()`, and `title()`?

- `upper()` converts all alphabetic characters in the string to uppercase.

- `lower()` converts all alphabetic characters to lowercase.
- `title()` capitalizes the first letter of every individual word in the string.

12. How can you reverse a string in Python?

You can easily reverse a string using extended slice syntax with a step of `-1`: `reversed_str = my_string[::-1]`.

13. How do you check whether a string is a palindrome?

A string is a palindrome if it reads the same forward and backward. You can check this by comparing the string to its reversed version: `is_palindrome = (my_string == my_string[::-1])`.

14. What is string interning?

String interning is an optimization technique where Python caches small or identical immutable string objects in memory. Instead of creating a new object for duplicate strings, Python makes them point to the exact same memory address, saving memory and speeding up comparison operations.

15. What is the difference between `isalpha()`, `isdigit()`, and `isalnum()`?

- `isalpha()` returns `True` if all characters in the string are alphabetic letters (A-Z, a-z) and the string is not empty.
- `isdigit()` returns `True` if all characters are digits (0-9).
- `isalnum()` returns `True` if all characters are alphanumeric (either letters or digits).

---

4. Lists, Tuples, Sets, and Dictionaries (51–75)

1. What is a list in Python?

A list is an ordered, mutable collection of items enclosed in square brackets (`[]`). It can store heterogeneous data types (integers, strings, floats, even other lists) and allows duplicate elements.

2. What is a tuple in Python?

A tuple is an ordered, immutable collection of items enclosed in parentheses `()`. Like lists, tuples can store mixed data types and allow duplicate elements, but they cannot be modified after creation.

3. What is a set in Python?

A set is an unordered collection of unique items enclosed in curly braces `{}` or created using the `set()` constructor. Sets do not allow duplicate values and are mutable, but their elements must be hashable (immutable).

4. What is a dictionary in Python?

A dictionary is an unordered (ordered by insertion since Python 3.7+) collection of key-value pairs enclosed in curly braces `{}`. Keys must be unique and immutable, while values can be of any data type and can be duplicated.

5. What is the difference between a list and a tuple?

- **Mutability:** Lists are mutable (can be changed in place), whereas tuples are immutable.
- **Syntax:** Lists use square brackets `[]`, while tuples use parentheses `()`.
- **Memory & Performance:** Tuples are generally more memory-efficient and slightly faster than lists because of their immutability.

6. What is the difference between a list and a set?

- **Order:** Lists maintain the insertion order of elements, whereas sets are unordered.
- **Duplicates:** Lists allow duplicate elements, but sets automatically eliminate duplicates.
- **Access:** List items can be accessed by index (`my_list[0]`), whereas set items cannot be indexed because sets are unordered.

7. What is the difference between a set and a dictionary?

- **Structure:** A set stores individual unique elements, whereas a dictionary stores pairs of unique keys and their corresponding values.

- **Syntax:** A set looks like `{item1, item2}`, while a dictionary looks like `{key1: value1, key2: value2}`.

8. Why are tuples considered immutable?

Tuples are considered immutable because once they are created, you cannot add, remove, or change any of their elements. Python enforces this at the memory level, meaning tuples lack mutation methods like `append()` or item assignment syntax (`t[0] = 5`).

9. Can a tuple contain a list?

Yes. While the tuple itself is immutable (its container structure and the memory references it holds cannot change), if it holds a mutable object like a list inside it, that inner list *can* be modified in place.

10. Can a list be used as a dictionary key?

No. Dictionary keys must be hashable and immutable (such as strings, numbers, or tuples). Because lists are mutable, their contents can change, which would change their hash value and break the dictionary's lookup mechanism.

11. What is the difference between `append()` and `extend()`?

- `append()` adds its argument as a **single element** to the end of the list (even if the argument is another list).
- `extend()` iterates over its iterable argument and appends **each element individually** to the list.

12. What is the difference between `remove()`, `pop()`, and `del`?

- `remove()` searches for and deletes the **first occurrence of a specific value** from the list.
- `pop()` removes and returns an element at a **specific index** (defaults to the last element if no index is provided).
- `del` is a statement used to delete an element at a specific index or slice, or to delete the entire list variable from memory.

13. What is the difference between `sort()` and `sorted()`?

- `sort()` is a list method that sorts the list **in place** and returns `None` (modifying the original list).
- `sorted()` is a built-in function that takes any iterable, sorts it, and returns a **brand-new sorted list** without modifying the original iterable.

14. How do you remove duplicates from a list?

The quickest way is to convert the list to a set and back: `my_list = list(set(my_list))`. However, this can alter the original order. To remove duplicates while preserving order, you can use `list(dict.fromkeys(my_list))`.

15. What is list comprehension?

List comprehension is a compact and readable syntax for creating a new list by applying an expression to items in an existing iterable, optionally filtered by a condition.

- **Syntax:** `[expression for item in iterable if condition]`

16. What is dictionary comprehension?

Dictionary comprehension is a concise way to construct dictionaries dynamically using a loop and expressions.

- **Syntax:** `{key_expr: value_expr for item in iterable if condition}`

17. What is a nested list?

A nested list is a list that contains other lists as its elements. It is commonly used to represent multi-dimensional data structures like matrices or grids.

18. What is a nested dictionary?

A nested dictionary is a dictionary in which the values are themselves other dictionaries. It is used to represent hierarchical data structures, like JSON objects or user profiles with multiple sub-categories.

19. What is the difference between `dict[key]` and `dict.get(key)`?

- `dict[key]` looks up the key and returns its value, but raises a `KeyError` if the key does not exist.
- `dict.get(key)` looks up the key safely, returning `None` (or a custom default value if specified, like `dict.get(key, 'default')`) if the key is missing.

20. What are `keys()`, `values()`, and `items()`?

These are dictionary view methods:

- `keys()` returns a view object displaying all the keys in the dictionary.
- `values()` returns a view object displaying all the values.
- `items()` returns a view object displaying all key-value pairs as tuples `(key, value)`.

21. What is unpacking in Python?

Unpacking is the process of extracting elements from a sequence (like a list or tuple) or mapping and assigning them to multiple individual variables in a single line (e.g., `a, b, c = [1, 2, 3]`). You can also use the `*` operator for extended iterable unpacking.

22. What are set union, intersection, and difference?

- **Union (`|` or `.union()`):** Combines all unique elements from both sets.
- **Intersection (`&` or `.intersection()`):** Returns only the elements common to both sets.
- **Difference (or `.difference()`):** Returns elements present in the first set that are not present in the second set.

23. What is the difference between a shallow copy and a deep copy?

- **Shallow copy (`copy()`):** Creates a new container object, but populates it with references to the original nested objects inside. Modifying nested mutable objects will affect both.
- **Deep copy (`copy.deepcopy()`):** Recursively copies the container and all nested objects, creating a completely independent clone.

24. What is the difference between assignment and copying a list?

- **Assignment (`b = a`):** Simply creates a new reference pointing to the exact same list object in memory. Any change made via `b` will reflect in `a`.
- **Copying (`b = a.copy()`):** Creates a distinct, separate list object in memory, ensuring modifications to one do not affect the other.

25. What is dictionary insertion order in Python?

Starting from Python 3.7+, dictionaries officially guarantee that they maintain the **insertion order** of their keys. When you iterate over a dictionary, items are yielded in the exact sequence they were originally added.

---

5. Functions (76–90)

1. What is a function in Python?

A function is a block of organized, reusable code designed to perform a specific task. It helps avoid code repetition and makes programs modular. Functions are defined using the `def` keyword followed by the function name and parentheses.

2. What is the difference between parameters and arguments?

- **Parameters** are the variables listed in the function definition (the blueprint or placeholders).
- **Arguments** are the actual values or data passed into the function when it is called.

3. What is the difference between `print()` and `return`?

- `print()` simply displays output to the console for debugging or user visibility, and it evaluates to `None`.
- `return` sends a value back to the caller of the function, allowing the program to store, manipulate, or use that result in subsequent code.

4. What happens when a function has no return statement?

If a function does not have an explicit `return` statement, it implicitly returns `None` after finishing execution.

5. What are default arguments?

Default arguments are parameters that take a pre-defined default value if no corresponding argument is provided during the function call (e.g., `def greet(name="Guest"): ...`).

6. What are keyword arguments?

Keyword arguments are arguments passed to a function accompanied by their parameter names (e.g., `func(name="Alice", age=25}`). This allows arguments to be passed out of order because Python matches them by name rather than position.

7. What are positional arguments?

Positional arguments are standard arguments that must be passed to a function in the exact order (position) that the parameters are defined in the function signature.

8. What are `args` and `*kwargs`?

- `args` (non-keyword arguments) allows a function to accept a variable number of positional arguments, packaging them into a **tuple**.
- `*kwargs` (keyword arguments) allows a function to accept a variable number of keyword arguments, packaging them into a **dictionary**.

9. What is a lambda function?

A lambda function is a small, anonymous (nameless) function defined using the `lambda` keyword. It can take any number of arguments but can only contain a **single expression**, which is evaluated and returned automatically.

10. What is recursion?

Recursion is a programming technique where a function calls **itself** to solve smaller instances of the same problem. Every recursive function requires a **base case** to stop the recursion and prevent infinite loops (and a `RecursionError`).

11. What is a higher-order function?

A higher-order function is a function that can take one or more functions as arguments, return a function as its result, or both. Built-in examples include `map()`, `filter()`, and `sorted()`.

12. What is a closure in Python?

A closure is a nested function that retains access to variables in its enclosing (outer) scope, even after the outer function has finished executing and returned.

13. What is a decorator?

A decorator is a design pattern in Python used to modify or extend the behavior of a function or method without permanently changing its source code. It takes a function as input, adds extra functionality, and returns a new function (often using the `@decorator_syntax`).

14. What is variable scope in Python?

Variable scope defines the region of a program where a variable is accessible. Python determines scope using the **LEGB** rule:

- **L**ocal (inside the current function)
- **E**nclosing (inside enclosing functions, like closures)
- **G**lobal (at the top level of the module)
- **B**uilt-in (predefined Python names like `print` or `len`)

15. What is the difference between `global` and `nonlocal`?

- `global` is used inside a function to declare that a variable refers to the global scope, allowing you to modify global variables from within a function.
- `nonlocal` is used inside a nested function to modify a variable in the nearest enclosing (outer) scope that is not global.

---

6. Object-Oriented Programming (91–110)

1. What is object-oriented programming?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **"objects,"** which bundle data (attributes) and behaviors (methods) together. It models real-world entities to make code more modular, reusable, and easier to maintain.

2. What is a class in Python?

A class is a user-defined blueprint or prototype from which objects are created. It defines a set of attributes and methods that the created objects will possess.

3. What is an object in Python?

An object is an instance of a class. When a class is instantiated, an object is created with allocated memory, possessing the specific attributes and methods defined by the class.

4. What is the difference between a class and an object?

- **Class:** A blueprint, template, or logical definition (e.g., a blueprint for a car).
- **Object:** An actual physical instance of that class (e.g., a specific red Honda Civic built from that blueprint).

5. What is the purpose of `self`?

`self` represents the specific instance of the class currently being created or interacted with. It allows access to the instance's attributes and methods. When you call an instance method, Python automatically passes the object itself as the first argument (`self`).

6. What is the purpose of `__init__()`?

`__init__()` is a special method known as a constructor in Python. It is automatically called when a new object of a class is instantiated, and its primary purpose is to initialize the object's attributes with starting values.

7. What are instance variables?

Instance variables are attributes that are unique to each individual object instance. They are typically defined inside the `__init__()` method using `self.variable_name`.

8. What are class variables?

Class variables are attributes shared across **all** instances of a class. They are defined directly inside the class body (outside any methods) and are useful for storing data that should be common to every object.

9. What are instance methods?

Instance methods are functions defined inside a class that operate on specific instances. They take `self` as their first parameter and can modify or access instance state.

10. What are class methods?

Class methods are bound to the class rather than its instances. They are decorated with `@classmethod` and take `cls` (referring to the class itself) as their first parameter, allowing them to modify class-level state.

11. What are static methods?

Static methods do not take `self` or `cls` as implicit arguments. They are decorated with `@staticmethod` and act like normal utility functions inside a class namespace, unable to modify class or instance state.

12. What are the four pillars of OOP?

The four core pillars of OOP are:

1. **Encapsulation**
2. **Abstraction**
3. **Inheritance**
4. **Polymorphism**

13. What is encapsulation?

Encapsulation is the bundling of data (attributes) and methods that operate on that data within a single unit (class), while restricting direct access to some of an object's components. In Python, this is achieved using private/protected naming conventions like leading underscores (`_protected` or `__private`).

14. What is abstraction?

Abstraction is the concept of hiding complex implementation details and showing only the essential features of an object. In Python, this is typically implemented using abstract base classes (`abc` module) and abstract methods.

15. What is inheritance?

Inheritance is a mechanism that allows a new class (child/derived class) to adopt the attributes and methods of an existing class (parent/base class), promoting code reusability.

16. What are the different types of inheritance?

1. **Single Inheritance:** A child class inherits from one parent class.
2. **Multilevel Inheritance:** A class inherits from another derived class, forming a chain (A → B → C).
3. **Hierarchical Inheritance:** Multiple child classes inherit from one parent class.
4. **Multiple Inheritance:** A child class inherits from two or more parent classes.
5. **Hybrid Inheritance:** A combination of two or more types of inheritance.

17. What is multiple inheritance?

Multiple inheritance occurs when a child class inherits from two or more parent classes simultaneously.

Example:

```
class Child(Parent1, Parent2):
    pass
```

Python uses the **Method Resolution Order (MRO)** to determine the order in which methods and attributes are searched. Python uses the C3 linearization algorithm to calculate the MRO.

18. What is polymorphism?

Polymorphism means "many forms." It allows different classes to use methods with the same name but different implementations, providing a common interface for different objects.

19. What is the difference between method overloading and method overriding?

- **Method Overriding:** A child class provides its own implementation of a method already defined in its parent class.
- **Method Overloading:** Multiple methods have the same name but different parameter lists within the same class.

Python does not natively support traditional method overloading based on parameter signatures. Similar behaviour can be achieved using default arguments, `*args`, and `**kwargs`.

20. What is the purpose of `super()`?

`super()` is a built-in function that returns a proxy object used to access methods according to the Method Resolution Order (MRO).

It is commonly used in `__init__()` to call parent class initialization logic.

Example:

```
class Parent:
    def __init__(self):
        print("Parent constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child constructor")

obj = Child()
```

Output:

```
Parent constructor
```

```
Child constructor
```

7. Exception Handling, Files, and Modules (111–125)

1. What is exception handling in Python?

Exception handling is a mechanism used to respond to runtime errors or unexpected events gracefully. Instead of letting a program crash abruptly, exception handling allows the code to catch the error, execute alternative logic, and keep the program running or terminate safely.

2. What is the difference between an error and an exception?

- **Errors** are severe, unrecoverable problems (like a `SyntaxError` or hardware-level `MemoryError`) that usually indicate critical issues with code structure or system state.
- **Exceptions** are abnormal conditions that occur during execution (like `ZeroDivisionError` or `FileNotFoundError`) that can be anticipated, caught, and handled by the programmer.

3. What is the purpose of `try` and `except`?

- `try` block contains code that might raise an exception.
- `except` block defines how the program should respond if a specific exception is raised inside the corresponding `try` block.

4. What is the purpose of `else` in exception handling?

The `else` block runs **only if the `try` block executes successfully without raising any exceptions**. It is useful for code that should only run when no errors occurred, keeping the `try` block concise.

5. What is the purpose of `finally`?

The `finally` block **always runs**, regardless of whether an exception was raised, caught, or ignored. It is typically used for cleanup actions, such as closing database connections, releasing locks, or closing files.

6. What is the purpose of the `raise` keyword?

The `raise` keyword is used to explicitly trigger or throw an exception in your code when a specific condition or business rule is violated.

7. How do you create a custom exception?

You create a custom exception by defining a new class that inherits from Python's built-in `Exception` base class (or a more specific exception class):

Python

```
class InvalidAgeError(Exception):
    pass
```

8. What is the difference between `SyntaxError`, `TypeError`, and `ValueError`?

- `SyntaxError`: Occurs when the code violates Python's grammar rules and cannot be parsed.
- `TypeError`: Occurs when an operation or function is applied to an object of an inappropriate type (e.g., adding a string and an integer).
- `ValueError`: Occurs when a function receives an argument of the correct type but an inappropriate value (e.g., trying to convert the string `"hello"` to an integer using `int()`).

9. What are the different file opening modes in Python?

- `'r'`: Read mode (default; opens a file for reading, raises an error if it doesn't exist).
- `'w'`: Write mode (opens for writing, overwrites existing content or creates a new file).
- `'a'`: Append mode (opens for writing, adding new data to the end of the file without erasing old content).
- `'x'`: Exclusive creation mode (creates a new file, fails if the file already exists).

- Modifiers: `'b'` for binary mode (e.g., `'rb'`) and `'+'` for updating (reading and writing).

10. What is the difference between `read()`, `readline()`, and `readlines()`?

- `read(size)` reads the entire file (or up to `size` bytes) as a single string.
- `readline()` reads a single line from the file.
- `readlines()` reads all lines in the file and returns them as a **list of strings**.

11. What is the purpose of the `with` statement when working with files?

The `with` statement acts as a context manager. It ensures that system resources—such as file handles—are **automatically and properly closed** as soon as the block of code inside the `with` statement exits, even if an exception is raised halfway through.

12. What is the difference between a module and a package?

- **A module** is a single Python file (ending in `.py`) containing variables, functions, and classes.
- **A package** is a directory (folder) containing multiple modules and a special file (traditionally `__init__.py`) that organizes modules into a hierarchical namespace.

13. What is the purpose of `import` in Python?

The `import` statement loads code from external modules or packages into the current namespace, making their functions, classes, and variables accessible in your script.

14. What is the purpose of `if __name__ == "__main__":`?

This idiom checks whether the script is being run directly by the Python interpreter or imported as a module into another script. If run directly, its special `__name__` variable is set to `"__main__"`, allowing the code inside the block to execute.

If imported, it stays the module's name, preventing test or execution code from running unintentionally upon import.

15. What is the difference between `pip` and a virtual environment?

- `pip` is Python's package manager used to download, install, and manage third-party libraries from the Python Package Index (PyPI).
- **A virtual environment** is an isolated folder containing its own private copy of the Python interpreter and site-packages, allowing developers to manage project-specific dependencies without conflicts.

---

8. Advanced Python (126–140)

1. What is an iterable in Python?

An iterable is any Python object that can be looped over (i.e., you can get an iterator from it). Examples include lists, tuples, strings, dictionaries, and sets. Technically, an iterable is any object that implements the `__iter__()` method or the `__getitem__()` method.

2. What is an iterator in Python?

An iterator is an object that represents a stream of data and keeps track of its current position during iteration. It implements two main methods:

- `__iter__()`: Returns the iterator object itself.
- `__next__()`: Returns the next item in the stream. When there are no more items left, it raises a `StopIteration` exception.

3. What is the difference between an iterable and an iterator?

- **Iterable:** Can be iterated over (e.g., a list), but it doesn't keep track of state on its own. You can call `iter()` on an iterable to get an iterator.
- **Iterator:** Keeps track of its current position and produces the next value when `__next__()` is called. **All iterators are iterables, but not all iterables are**

iterators (for instance, a list is an iterable, but it is not an iterator until you pass it to `iter()`).

4. What is a generator in Python?

A generator is a special type of function that returns an iterator using the `yield` keyword instead of `return`. Instead of computing and storing an entire sequence in memory at once, a generator produces values **lazily** (on the fly), making it extremely memory-efficient for large datasets.

5. What is the difference between `yield` and `return`?

- `return` sends a value back to the caller and completely terminates the execution of the function.
- `yield` pauses the function, saves its internal state, sends a value back to the caller, and allows the function to resume right where it left off when the next value is requested.

6. What is a generator expression?

A generator expression is a concise way to create an anonymous generator on the fly using syntax similar to a list comprehension, but enclosed in **parentheses** `()` instead of square brackets `[]`. It yields items one by one rather than building a full list in memory.

7. What are magic or dunder methods?

Magic or "dunder" (double underscore) methods are special methods with names starting and ending with double underscores (e.g., `__init__`, `__str__`, `__len__`). They allow developers to define or hook into Python's built-in behaviors and operators for custom classes.

8. What is the difference between `__str__()` and `__repr__()`?

- `__str__()`: Intended to provide an informal, user-friendly string representation of an object (called by `print()` or `str()`).
- `__repr__()`: Intended to provide an unambiguous, formal representation of an object (often used for debugging and development), which ideally can be used to recreate the object via `eval()`.

9. What is operator overloading?

Operator overloading is the ability to redefine how standard operators (like `+`, `-`, `*`, `==`) behave when applied to user-defined class instances. This is achieved by implementing the corresponding dunder methods (e.g., `__add__` for `+`, `__eq__` for `==`).

10. What is a context manager?

A context manager is an object that defines the runtime context for a `with` statement. It ensures proper resource setup and teardown by implementing two dunder methods:

- `__enter__()`: Sets up the resource and returns it.
- `__exit__()`: Handles cleanup operations (like closing a file or releasing a lock) when the block exits, even if an exception occurs.

11. How does Python manage memory?

Python manages memory through a private heap space that holds all Python objects and data structures. It uses a built-in memory manager, **reference counting**, and a **garbage collector** to dynamically allocate memory and deallocate it when objects are no longer in use.

12. What is garbage collection?

Garbage collection is the automated process of freeing up memory by destroying objects that are no longer referenced by the program. Python's primary mechanism is **reference counting** (deleting an object the moment its reference count hits zero), backed up by a **generational cyclic garbage collector** that detects and cleans up complex reference cycles (e.g., two objects referencing each other).

13. What is the Global Interpreter Lock (GIL)?

The GIL is a mutex (lock) used by CPython (the standard Python interpreter) that ensures **only one native thread executes Python bytecode at a time**. While it protects memory safety and makes single-threaded programs fast, it prevents true CPU-bound multi-threaded parallelism on multi-core processors.

14. What is the difference between multithreading and multiprocessing?

- **Multithreading:** Runs multiple threads within a single process, sharing the same memory space. Best suited for **I/O-bound tasks** (like downloading web pages or reading files). In Python, it is limited by the GIL for CPU-bound tasks.
- **Multiprocessing:** Spawns separate processes, each with its own independent memory space and Python interpreter. It completely **bypasses the GIL**, making it ideal for **CPU-bound tasks** (like heavy mathematical computations) across multiple CPU cores.

15. What is the difference between shallow copying and deep copying of nested objects?

- **Shallow copy (`copy.copy()`):** Creates a new outer container object, but populates it with references to the original nested objects. Modifying a mutable nested object inside a shallow copy will affect the original object.
- **Deep copy (`copy.deepcopy()`):** Recursively clones the container and **all nested objects**, creating a completely independent copy where changes to nested structures have no effect on the original.

---

9. Coding and Practical Questions (141–150)

1. Program to reverse a string

Python

```
def reverse_string(s):
  return s[::-1]


# Example usage:
print(reverse_string("python"))  # Output: nohtyp
```

2. Program to check whether a number is prime

Python

```python
def is_prime(n):
  if n <= 1:
    return False
  for i in range(2, int(n**0.5) + 1):
    if n % i == 0:
      return False
  return True


# Example usage:
print(is_prime(11))  # Output: True
print(is_prime(4))  # Output: False
```

3. Program to check whether a string is a palindrome

Python

```python
def is_palindrome(s):
  # Remove spaces/punctuation and convert to lowercase for ac
curate check
  clean_s = "".join(c.lower() for c in s if c.isalnum())
  return clean_s == clean_s[::-1]


# Example usage:
print(is_palindrome("Racecar"))  # Output: True
print(is_palindrome("Hello"))  # Output: False
```

4. Program to find the factorial of a number

Python

```python
def factorial(n):
  if n < 0:
    return "Factorial does not exist for negative numbers."
```

```
  result = 1
  for i in range(2, n + 1):
    result *= i
  return result


# Example usage:
print(factorial(5))  # Output: 120
```

5. Program to generate the Fibonacci series

Python

```
def generate_fibonacci(n):
  if n <= 0:
    return []
  elif n == 1:
    return [0]
  fib = [0, 1]
  while len(fib) < n:
    fib.append(fib[-1] + fib[-2])
  return fib


# Example usage:
print(generate_fibonacci(7))  # Output: [0, 1, 1, 2, 3, 5, 8]
```

6. Program to find the second-largest element in a list

Python

```
def second_largest(nums):
  unique_nums = list(set(nums))
  if len(unique_nums) < 2:
    return None
  unique_nums.sort()
```

```
  return unique_nums[-2]


# Example usage:
print(second_largest([10, 20, 4, 45, 99, 99]))  # Output: 45
```

7. Program to count the frequency of each character in a string

Python

```
def char_frequency(s):
  freq = {}
  for char in s:
    freq[char] = freq.get(char, 0) + 1
  return freq


# Example usage:
print(char_frequency("banana"))  # Output: {'b': 1, 'a': 3,
'n': 2}
```

8. Program to remove duplicates from a list without using a set

Python

```
def remove_duplicates(nums):
  unique_list = []
  for item in nums:
    if item not in unique_list:
      unique_list.append(item)
  return unique_list


# Example usage:
```

```
print(remove_duplicates([1, 2, 2, 3, 4, 1, 5]))  # Output:
[1, 2, 3, 4, 5]
```

9. Program to check whether two strings are anagrams

Python

```
def are_anagrams(s1, s2):
  # Normalize by removing spaces and converting to lowercase
  s1 = s1.replace(" ", "").lower()
  s2 = s2.replace(" ", "").lower()
  return sorted(s1) == sorted(s2)


# Example usage:
print(are_anagrams("Listen", "Silent"))  # Output: True
```

10. Python program to connect to MySQL and perform CRUD operations

(Requires the `mysql-connector-python` package installed via pip)

Python

```
import mysql.connector


def perform_mysql_crud():
  try:
    # 1. Establish Connection
    connection = mysql.connector.connect(
        host="localhost",
        user="your_username",
        password="your_password",
        database="your_database",
    )
    cursor = connection.cursor()
```

```
    # --- CREATE (Insert Record) ---
    insert_query = (
        "INSERT INTO employees (name, department) VALUES (%s,
%s)"
    )
    cursor.execute(insert_query, ("Alice Smith", "Engineerin
g"))
    connection.commit()
    print("Record inserted successfully.")

    # --- READ (Select Records) ---
    cursor.execute("SELECT * FROM employees")
    print("\nReading records:")
    for row in cursor.fetchall():
      print(row)

    # --- UPDATE (Modify Record) ---
    update_query = (
        "UPDATE employees SET department = %s WHERE name = %
s"
    )
    cursor.execute(update_query, ("Data Science", "Alice Smit
h"))
    connection.commit()
    print("\nRecord updated successfully.")

    # --- DELETE (Remove Record) ---
    delete_query = "DELETE FROM employees WHERE name = %s"
    cursor.execute(delete_query, ("Alice Smith",))
    connection.commit()
    print("\nRecord deleted successfully.")

  except mysql.connector.Error as err:
    print(f"Error: {err}")
```

```
  finally:
    # Close cursor and connection
    if "connection" in locals() and connection.is_connected
():
      cursor.close()
      connection.close()
      print("MySQL connection closed.")


# perform_mysql_crud()
```